

lab2_dqn_torch

October 23, 2025

```
[1]: %reload_ext autoreload
      %autoreload 2
```

```
[2]: from kret_studies import *
      from kret_studies.notebook import *
      from kret_studies.complex import *
      from kret_studies.openai_gym import *

      logger = get_notebook_logger()
```

Loaded environment variables from
/Users/Akseldkw/Desktop/Columbia/ORCS4529/.env.

INFO:datasets:JAX version 0.7.2 available.
INFO:arviz.preview:arviz_base not installed
INFO:arviz.preview:arviz_stats not installed
INFO:arviz.preview:arviz_plots not installed

/Users/Akseldkw/coding/kretsinger/data/nb_log.log

```
[3]: env_vars = os.environ.copy()
      wand_db_dir = env_vars["OUTPUT_DIR"] + "/wandb"
```

0.1 Delete above

```
[4]: from gymnasium.wrappers.common import TimeLimit
      from gymnasium.spaces import Discrete, Box
      import typing as t
      from collections import deque
      from numpy.typing import NDArray
      import random
```

```
[5]: import torch
      from collections import deque

      import numpy as np

      %matplotlib inline
```

```
import matplotlib
import matplotlib.pyplot as plt

# import gym
```

```
[6]: # !pip install wandb -qqq
import wandb

wandb.login()
run = wandb.init(dir=wand_db_dir, project="orcs4529_hw2_dqn")
```

```
/Users/Akseldkw/micromamba/envs/kret_312/lib/python3.12/site-
packages/pydantic/_internal/_generate_schema.py:2249:
UnsupportedFieldAttributeWarning: The 'repr' attribute with value False was
provided to the `Field()` function, which has no effect in the context it was
used. 'repr' is field-specific metadata, and can only be attached to a model
field using `Annotated` metadata or by assignment. This may have happened
because an `Annotated` type alias using the `type` statement was used, or if the
`Field()` function was attached to a single member of a union type.
```

```
warnings.warn(
/Users/Akseldkw/micromamba/envs/kret_312/lib/python3.12/site-
packages/pydantic/_internal/_generate_schema.py:2249:
UnsupportedFieldAttributeWarning: The 'frozen' attribute with value True was
provided to the `Field()` function, which has no effect in the context it was
used. 'frozen' is field-specific metadata, and can only be attached to a model
field using `Annotated` metadata or by assignment. This may have happened
because an `Annotated` type alias using the `type` statement was used, or if the
`Field()` function was attached to a single member of a union type.
```

```
warnings.warn(
wandb: Currently logged in as: akseldkw
(akseldkw07) to https://api.wandb.ai. Use `wandb login
```

```
--relogin` to force relogin
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

0.2 DQN (Deep Q Network)

In previous Labs, we have learned to use Pytorch to build deep learning models. In this lab, we will apply deep learning as function approximations in reinforcement learning.

Reference: DQN <https://arxiv.org/abs/1312.5602>

In tabular Q-learning, we maintain a table of state-action pairs (s, a) and save one action value for each entry $Q(s, a), \forall (s, a)$. At each time step t , we are in state s_t , then we choose action based on ϵ -greedy strategy. With prob ϵ , choose action uniformly random; with prob $1 - \epsilon$, choose action based on

$$a_t = \arg \max_a Q(s_t, a)$$

We then get the instant reward r_t , update the Q-table using the following rule

$$Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha(r_t + \max_a \gamma Q(s_{t+1}, a))$$

where $\alpha \in (0, 1)$ is learning rate. The algorithm is shown to converge in tabular cases. However, in cases where we cannot keep a table for state and action, we need function approximation. Consider using neural network with parameter θ , the network takes as input state s and action a . (*there are alternative parameterizations here*). Let $Q_\theta(s, a)$ be the output of the network, to estimate the optimal action value function in state s and take action a (and follow optimal policy thereafter).

$$Q_\theta(s, a) \approx Q^*(s, a)$$

0.2.1 Bellman optimality equation

We will use Bellman optimality equation to find θ such that the above approximation holds better. Recall that for optimal Q function $Q^*(s, a)$ the following holds for all (s, a)

$$Q^*(s_t, a_t) = \mathbb{E}[r_t + \gamma \max_a Q^*(s_{t+1}, a)]$$

where the expectation is wrt the random reward r_t and transition to the next state s_{t+1} . A natural objective to consider is

$$\min_{\theta} (Q_\theta(s_t, a_t) - \mathbb{E}[r_t + \gamma \max_a Q_{\hat{\theta}}(s_{t+1}, a)])^2$$

at the current or previous $\hat{\theta}$.

0.2.2 Building the DQN model

The first step is to build a neural network with parameters θ that predicts $Q_\theta(s, a)$ for any (s, a) . You can either build a network that

- (in case of small number K of discrete actions) takes as input a representation of state s and outputs a K -dimensional vector giving scores $Q(s, a), a = 1, \dots, K$ for all actions

or

- takes as input a concatenated representation of state and action (s, a) and output one dimensional score $Q_\theta(s, a)$,

Below we have provided a skeleton code (incomplete) for defining and training the Q-function. **You need to fill in the DNN model definition and loss function definition.** Refer to regression lab (lab 2) for help.

```

[17]: # define neural net  $Q_\theta(s,a)$  as a class

class Qfunction(object):
    obssize: int
    actsize: int
    h1: int
    h2: int
    model: torch.nn.Module
    optimizer: torch.optim.Optimizer

    def __init__(self, obssize: int, actsize: int | np.signedinteger, lr: float):
        """
        obssize: dimension of state space
        actsize: dimension of action space
        sess: sess to execute this Qfunction
        optimizer:
        """
        # RECORD HYPER-PARAMS
        self.obssize = obssize
        self.actsize = int(actsize)

        self.h1 = min(512, max(64, 2 * obssize))
        self.h2 = min(512, max(64, int(obssize + actsize)))

        # DEFINE THE MODEL
        self.model = torch.nn.Sequential(
            torch.nn.Linear(self.obssize, self.h1),
            torch.nn.ReLU(),
            torch.nn.Linear(self.h1, self.h2),
            torch.nn.ReLU(),
            torch.nn.Linear(self.h2, self.actsize), # outputs  $Q(s,a)$  for all  $a$ 
        )

        # DEFINE THE OPTIMIZER
        self.optimizer = torch.optim.Adam(self.model.parameters(), lr=lr)

    def _to_one_hot(self, y: torch.Tensor, num_classes: int):
        """
        convert an integer vector  $y$  into one-hot representation
        """
        scatter_dim = len(y.size())
        y_tensor = y.view(*y.size(), -1)
        zeros = torch.zeros(*y.size(), num_classes, dtype=y.dtype)
        return zeros.scatter(scatter_dim, y_tensor, 1)

```

```

        y = y.view(-1, 1).long()
        out = torch.zeros(y.size(0), num_classes, dtype=torch.float32, device=y.
↪device)
        out.scatter_(1, y, 1.0)
        return out

def compute_Qvalues(
    self, states: torch.Tensor, actions: torch.Tensor
) -> torch.Tensor:
    """
    input: list of numsamples state-action pairs
    output: List of Q values for each input (s,a). The output will have ↪
↪size [numsamples, 1]
    """
    # Below is example code when neural network is set to take as input ↪
↪state and output Q-value for all actions.
    # This will be different for neural network that takes as input a ↪
↪state-action pair

    states = states.float().view(-1, self.obssize) # (N, 4)
    actions = actions.view(-1, 1).long() # (N, 1)
    q_all = self.model(states) # (N, 2)
    return q_all.gather(1, actions).squeeze(1)

def compute_maxQvalues(self, states: torch.Tensor) -> torch.Tensor:
    """
    input: a list of numsamples states
    output: max_a Q(s,a) values for every input state s in states. The ↪
↪output will have size numsamples
    """
    # Below is example code when neural network is set to take as input ↪
↪state and output Q-value for all actions.
    # if the neural takes as input a state-action pair, then the code will ↪
↪need to loop over all actions to compute all values

    states = states.float().view(-1, self.obssize) # <- force (N, 4)
    q_all = self.model(states) # (N, actsize)
    return torch.max(q_all, dim=1).values

def compute_argmaxQ(
    self, state: np.ndarray | torch.Tensor, epsilon: float
) -> np.int64:
    """
    input:
        state: (obssize,) or (1, obssize)
    output:

```

```

        greedy action index as np.int64
        """
        if isinstance(state, np.ndarray):
            st = torch.from_numpy(state).float()
        else:
            st = state.float()

        if st.dim() == 1:
            st = st.unsqueeze(0)  # (1, S)

        if np.random.rand() < epsilon:
            a = np.random.randint(self.actsize)
        else:
            q: torch.Tensor = self.model(st)  # (1, A)
            a: int = int(torch.argmax(q, dim=1).item())

        return np.int64(a)

    def train(
        self,
        states: np.ndarray | torch.Tensor,
        actions: np.ndarray | torch.Tensor,
        targets: np.ndarray | torch.Tensor,
    ):
        """
        states: numpy array as input to compute loss (s)
        actions: numpy array as input to compute loss (a)
        targets: numpy array as input to compute loss (Q targets)
        """
        states = torch.FloatTensor(states)
        actions = torch.LongTensor(actions)
        targets = torch.FloatTensor(targets)

        # COMPUTE Q PREDICTIONS for all state-action pairs
        q_preds_selected = self.compute_Qvalues(states, actions)

        # LOSS
        # print(q_preds_selected.shape, targets.shape)
        loss = torch.mean((q_preds_selected - targets) ** 2)

        # BACKWARD PASS
        self.optimizer.zero_grad()
        loss.backward()

        # UPDATE
        self.optimizer.step()

```

```
return loss.detach().cpu().data.numpy()
```

At this point you can skip ahead to implementing the basic Q-learning that at every step t in the environment * given state s_t , computes greedy actions from Q-values (using `compute_argmaxQ` function above) and uses ϵ -greedy select an action a_t , * makes observation of reward r_t and next state s_{t+1} * using `compute_maxQvalues()` function, computes target

$$r_t + \gamma \max_a Q_\theta(s_{t+1}, a)$$

and then retrains the Q-function using `train()` function above (with `numsamples=1`)

However, for improved performance you may want to consider ideas like batch training (`numsamples>1` is the batch size) with experience replay buffer and target-networks.

Replay Buffer

Maintain a buffer R to store transition tuples (s_t, a_t, r_t, s_{t+1}) , when we minimize the Bellman error. When optimizing the Bellman error loss, we sample batches from the replay buffer and compute gradients for update on these batches. In particular, in each update, we sample N tuples from buffer $(s_i, a_i, r_i, s'_i) \sim R$ and then compute targets

$$d_i = r_i + \max_a \gamma Q_\theta(s'_i, a)$$

for all i . Use the above training function `train()` with input as list $(s_i, a_i, d_i)_{i=1}^N$ to update parameters using `backprop`.

Target Network

Maintain a target network in addition to the original principal network. The target network is just a copy of the original network but the parameters are not updated by gradients. The target network θ^- is copied from the principal network every τ time steps. Target network is used to compute the targets for update

$$d_i = r_t + \gamma \max_a Q_{\theta^-}(s'_i, a)$$

the targets are used in the loss function to update the principal network parameters. This slowly updated target network ensures that the targets come from a relatively stationary distribution and hence stabilize learning.

Hence several critical parts of the complete pseudocode for DQN is as follows:

Initialization. principal network $Q_\theta(s, a)$, target network $Q_{\theta^-}(s, a)$. Replay buffer $R = \{\}$ (empty).

At each time step t . The agent executes action using ϵ -greedy based on the principal network $Q_\theta(s, a)$. To update θ : sample N tuples $(s_i, a_i, r_i, s'_i) \sim R$, compute empirical loss

$$\frac{1}{N} \sum_{i=1}^N (Q_\theta(s_i, a_i) - (r_i + \gamma \max_a Q_{\theta^-}(s'_i, a)))^2$$

and update parameter θ using `backprop` (just take one gradient step).

Update target network. Every τ time steps, update target network by copying $\theta_{\text{target}} \leftarrow \theta$.

Bellman target. Above, we have defined the target values as being computed from a target net with parameter θ^-

$$r_i + \gamma \max_a Q_{\theta^-}(s'_i, a)$$

It is worth thinking about what happens if we are at the end of an episode, that is, what if s'_i here is a terminal state. In this case, should the Bellman error be defined exactly the same as above? Do we need some modifications? Think carefully about this as this will greatly impact the algorithmic performance.

0.2.3 Implementation of replay buffer

```
[18]: from functools import cache
      from typing import SupportsFloat

      @cache
      def get_weights(size: int, decay_denom: int):
          x = np.arange(1, size + 1)
          unweighted = np.exp((x - size) / decay_denom)
          return unweighted

      Transition = tuple[
          np.ndarray,
          int | np.signedinteger,
          float | SupportsFloat,
          t.Literal[0, 1],
          np.ndarray,
      ]

      class ReplayBuffer(deque[Transition]):
          x: NDArray[np.int_]
          idxs_last: NDArray[np.int_] # indexes of last sampled batch

          def __init__(self, maxlen: int):
              super().__init__(maxlen=maxlen)

          def sample_batch(self, batch_size: int, decay_denom: int | None = None):
              unweighted = get_weights(self.maxlen, decay_denom or self.maxlen)[0 :
↳ len(self)]
              weights = unweighted / np.sum(unweighted)

              batch_size = min(batch_size, len(self))
              self.idxs_last = np.random.choice(
                  len(self), batch_size, replace=False, p=weights
```



```

    )

    # return [self[i] for i in idxs]
    s, a, r, s2, d = zip(*(self[i] for i in self.idxs_last))
    return (
        np.stack(s),
        np.asarray(a),
        np.asarray(r, dtype=np.float32),
        np.stack(s2),
        np.asarray(d, dtype=np.bool_),
    )

```

0.2.4 Code snippet for copying target network

You may use the following to update target network i.e. to copy from principal network to target network. We need to use tensorflow scope to distinguish the computational graphs of target and principal networks. The following function builds a tensorflow operation that does the copying $\theta^- \leftarrow \theta$

```

[19]: def run_target_update(Qprincipal: Qfunction, Qtarget: Qfunction):
        for v, v_ in zip(Qprincipal.model.parameters(), Qtarget.model.parameters()):
            v_.data.copy_(v.data)

```

0.3 Main code for DQN

Now that we have all the ingredients for DQN, we can write the main procedure to train DQN on a given environment. The implementation is straightforward if you follow the pseudocode pdf.

```

[20]: envname = "CartPole-v1" # environment name
env = t.cast(TimeLimit[NDArray, np.int64], gym.make(envname))
type(env)

```

```

[20]: gymnasium.wrappers.common.TimeLimit

```

```

[21]: x = np.arange(1, 1000)
y2 = np.exp2((x - 1000) / 600)
y10 = np.exp(-(1000 - x) / 1000)

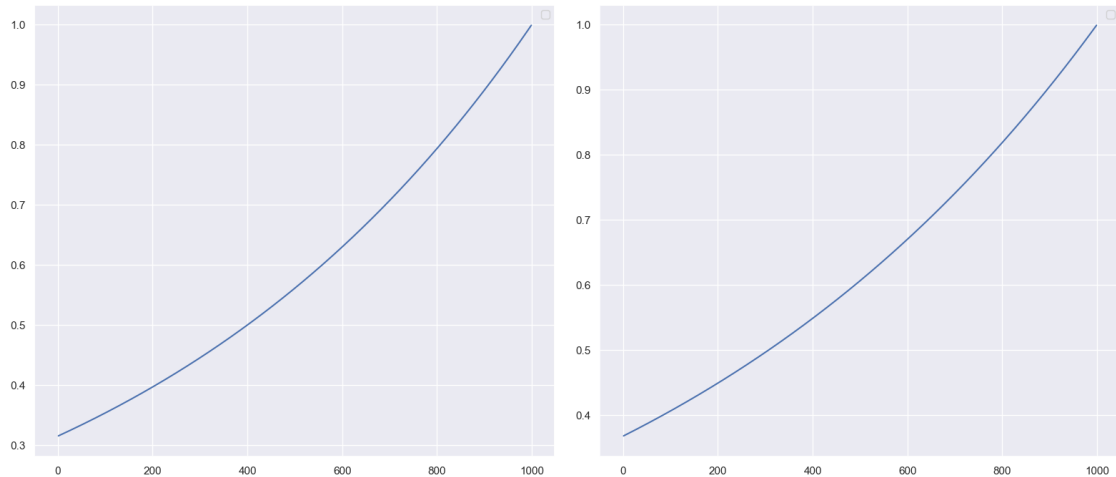
fig, ax = uks_mpl.subplots(2, 1)
ax[0].plot(x, y2)
ax[1].plot(x, y10)
fig

```

```

[21]:

```



```
[22]: obs = env.reset()[0]
```

0.4 Loop

```
[23]: # %%wandb
# remove above line if you do not want to see inline plots from wandb

# hyper-parameters
lr = 1e-3 # learning rate for gradient update
batchsize = 64 # batchsize for buffer sampling
maxlength = 1000 # max number of tuples held by buffer
envname = "CartPole-v0" # environment name
tau = 100 # time steps for target update
episodes = 300 # number of episodes to run
initialsize = 500 # initial time steps before start training
trainfreq = 20 # frequency of training steps
epsilon = 0.2 # constant for exploration
gamma = 0.99 # discount

# initialize environment
# env: TimeLimit = gym.make(envname)
env.observation_space = t.cast(Box, env.observation_space)
env.action_space = t.cast(Discrete, env.action_space)
obssize = t.cast(Box, env.observation_space).low.size
actsize = t.cast(Discrete, env.action_space).n

# initialize Q-function networks (principal and target)
Qprincipal = Qfunction(obssize, actsize, lr)
Qtarget = Qfunction(obssize, actsize, lr)
```

```

# initialization of graph and buffer
buffer = ReplayBuffer(maxlength)

# main iteration
rrecord = []
totalstep = 0

for episode in range(episodes):

    obs, info = env.reset()
    done = False
    rsum = 0

    while not done:

        # greedy choice below. Use epsilon greedy for exploration
        action = Qprincipal.compute_argmaxQ(np.expand_dims(obs, 0),
        ↪epsilon=epsilon)

        newobs, r, done, _, info = env.step(action)
        done_ = 1 if done else 0
        e = (obs, action, r, done_, newobs)

        # IF NOT USING BUFFER:
        # use single sample (obs, action, r, done_, newobs) with Qtarget to
        ↪compute target and train Qprincipal

        # ELSE IF USING REPLAY BUFFER
        # append experiences e to buffer
        buffer.append(e)

        # every few episodes (decide the frequency) sample a minibatch from
        ↪buffer

        # compute targets in batch using Qtarget and train Qprincipal
        if totalstep > initialsize and totalstep % trainfreq == 0:
            states, actions, rewards, newstates, dones = buffer.sample_batch(
                batchsize, decay_denom=maxlength
            )

            # compute targets
            qnews = Qtarget.compute_maxQvalues(torch.FloatTensor(newstates)).
            ↪detach()

            targets = rewards + gamma * qnews * (1 - dones)

            # train Qprincipal
            loss = Qprincipal.train(states, actions, targets)

```

```

        # UPDATE target network
        # every tau steps update copy the principal network to the target
↪network
        if totalstep % tau == 0:
            run_target_update(Qprincipal, Qtarget)

        # update
        totalstep += 1
        rsum += r
        obs = newobs

    # The code below is for printing and debugging at the end of episode

    rrecord.append(rsum)

    # printing functions for debugging purposes. Feel free to add more
    if episode % 10 == 0:
        print("buffer size {}".format(len(buffer)))
        print(
            "episode {} ave training returns {}".format(episode, np.
↪mean(rrecord[-10:])))
        )

    # printing moving averages for smoothed visualization.
    fixedWindow = 100
    movingAverage = 0
    if len(rrecord) >= fixedWindow:
        movingAverage = np.mean(rrecord[len(rrecord) - fixedWindow :
↪len(rrecord) - 1])
        wandb.log({"training reward": rsum, "train reward moving average":
↪movingAverage})

```

```

buffer size 11
episode 0 ave training returns 11.0
buffer size 111
episode 10 ave training returns 10.0
buffer size 212
episode 20 ave training returns 10.1
buffer size 312
episode 30 ave training returns 10.0
buffer size 410
episode 40 ave training returns 9.8
buffer size 516
episode 50 ave training returns 10.6

```

ValueError

Traceback (most recent call last)

```

Cell In[23], line 65
    63 # compute targets
    64 qnews = Qtarget.compute_maxQvalues(torch.FloatTensor(newstates)).detach()
--> 65 targets = rewards + gamma * qnews * (1 - dones)
    67 # train Qprincipal
    68 loss = Qprincipal.train(states, actions, targets)

ValueError: operands could not be broadcast together with shapes (16,) (64,4)

```

Finally, we evaluate the performance of the trained agent. We will evaluate the performance of the greedy policy wrt learned Q-function. The evaluation will be run 10 times, each for eval_episodes and print out the average performance across these episodes. Please **do not** change the code below.

```

[ ]: ### DO NOT CHANGE
def evaluate(Q, env, episodes):
    # main iteration
    score = 0.0
    for episode in range(episodes):

        obs = env.reset()
        done = False
        rsum = 0

        while not done:
            # always greedy
            action = Q.compute_argmaxQ(np.expand_dims(obs, 0))

            # mdp stepping forward
            newobs, r, done, _ = env.step(action)

            # update data
            rsum += r
            obs = newobs

        wandb.log({"eval reward": rsum})
        score = score + rsum
    score = score / episodes

    return score

```

```

[ ]: # DO NOT CHANGE CODE HERE
# after training, we will evaluate the performance of the agent
# on a target environment
env_test = gym.make(envname)
eval_episodes = 1000
score = evaluate(Qprincipal, env_test, eval_episodes)
wandb.run.summary["score"] = score

```

```
print("eval performance of DQN agent: {}".format(score))
```

```
-----  
NameError                                Traceback (most recent call last)  
<ipython-input-8-4856245aab7e> in <cell line: 6>()  
      4 env_test = gym.make(envname)  
      5 eval_episodes = 1000  
----> 6 score = evaluate(Qprincipal, env_test, eval_episodes)  
      7 wandb.run.summary["score"]=score  
      8  
  
NameError: name 'Qprincipal' is not defined
```

```
[ ]: run.finish()
```